Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	newbies
Project / Solution Name	Veracity
Name (Leader)	KOH HUI YUAN
Email & Phone (Leader)	huiyuankoh0409@gmail.com / +6011-51306389
University Affiliation(s)	Tunku Abdul Rahman University of Management and Technology (TARUMT)
Live Prototype URL	https://sdcveracitynewbies-lowzh51.replit.app/login
Code Repository URL (public / access granted to judges)	https://github.com/khyuannn/sdc_veracity_newbies

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

1. Veracity uses a React, TypeScript and Tailwind CSS web interface for stock count submission, photo-count review, crate tracking, evidence review, manager approvals and Decision Passport history. The frontend sends requests to an Express/TypeScript gateway.
2. For inventory functions, Express forwards authenticated requests to a Python backend. The backend applies role checks, records physical counts and crate activity, calculates expected stock from recorded stock plus eligible unposted movements, evaluates deterministic evidence rules, and saves audit records, assessment snapshots and decisions in SQLite.
3. For photo-assisted item counting, the frontend sends a downsized JPG/PNG image, selected product name and staff-drawn exemplar box to POST /api/vision/count. By default, Express runs a local sharp-based template-matching counter. If RunPod credentials are configured, Express calls a RunPod Serverless GPU worker running the pre-trained OWLv2 object-detection model. Suggested boxes and quantities return to the user for review and confirmation; no AI result automatically changes stock.
4. RFID functionality is a simulated or entered-ID crate lookup workflow. It records crate tags, locations and transfers, but does not claim verified physical RFID hardware integration.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	***Purpose in Project***	***License***
TypeScript 5.9.3	*Type-safe frontend, Express gateway and shared API code.*	*Apache-2.0*
React 19.1.0 + React DOM 19.1.0	*Builds the web interface for inventory counts, crate tracking, approvals and reports.*	*MIT*
Vite 7.3.6 + Tailwind CSS 4.3.3	*Frontend development/build tooling and responsive interface styling.*	*MIT*
Express 5.3.0	*TypeScript API gateway; handles photo-count requests and forwards application requests to the Python backend.*	*MIT*
Sharp 0.35.5	*Processes uploaded photos and performs the built-in template-matching photo-count workflow.*	*Apache-2.0*
Python 3.10+ standard library + SQLite	*Backend reconciliation logic, crate records, audit data, sessions and persistent inventory storage.*	*PSF License / Public Domain*

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
RunPod Serverless API — RunPod	Sends a photo, product label and selected example box to the optional GPU worker; receives OWLv2 detection boxes and suggested count.	Pay-as-you-go / sponsor-provided credits if allocated
Replit App Storage / managed Google Cloud Storage integration — Replit	Stores private evidence attachments when cloud storage is configured.	Replit-managed plan / free tier where available

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	***Purpose in Project***	***Plan / Region***
Replit	*Hosts the React frontend, Express gateway and Python backend; can provide private App Storage for evidence.*	*Free / managed multi-region deployment*
RunPod Serverless	*Optional GPU compute for the OWLv2 photo-object-counting worker. It is not required because a local template-matching fallback exists.*	*Pay-as-you-go GPU / region selected when endpoint is created*

SQLite	*Stores inventory, movements, counts, crate records, scans, assessments, decisions, sessions and audit records.*	*Local embedded database; no cloud region*
Local development launcher	*Runs the complete prototype locally with SQLite and private local evidence storage.*	*Developer machine*

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
OWLv2 Base Patch16 Ensemble (google/owlv2-base-patch16-ensemble)	Google, distributed through Hugging Face	Pre-trained; used only when optional RunPod GPU configuration is enabled. No fine-tuning or self-training.	Apache-2.0

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Veracity product, movement and additional-evidence CSV fixtures	docs/test-data/evidence-pack/csv-records/ in the project repository	Synthetic	Team/project test data	N
Veracity evidence-upload files	docs/test-data/evidence-pack/upload-documents/ in the project repository	Synthetic	Team/project test data	N

Snack Biscuits 39 photo-count fixture	docs/test-data/photo-count/snack-biscuits-39.jpg in the project repository	Synthetic test fixture	Team/project test data	N
OWLv2 upstream training data	Refer to the OWLv2 model card	Public upstream data; not collected, controlled or used for fine-tuning by the team	Governed by the model provider's terms	N — no team-collected personal data
Team training/fine-tuning dataset	Not applicable	None; the team does not train or fine-tune a model	Not applicable	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	Used For	Extent of Use (minor / moderate / major)
ChatGPT / Codex *(version varies by session)*	Requirements interpretation, implementation drafts, debugging, API integration, tests and technical documentation.	Major
v0.app *(confirm version with original frontend contributor)*	Initial frontend prototype and UI component generation.	Major
GitHub Copilot *(include only if actually used by the team)*	Code completion and implementation assistance.	Moderate

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

The team reviews AI-assisted output against the implemented code, manually tests key workflows and runs automated backend, type-check and browser tests. Team members are responsible for understanding and explaining the inventory reconciliation calculation, the required staff review of photo-counting results, the limitations of simulated or entered-ID RFID input, role permissions and manager approval. AI-generated code or documentation is revised when it does not match the implementation. The team remains responsible for all submitted functionality, technical claims and final decisions.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Credits	Replit hosting and deployment environment for the web application.
Runpod	Credits	RunPod credits / RunPod Serverless GPU compute for the optional OWLv2 photo-counting worker.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

No. The core inventory reconciliation, crate tracking, evidence review and manager-approval workflow run without optional cloud AI services. If RunPod is unavailable, the application uses its built-in template-matching counter, and staff can enter a verified physical count manually. Replit is a supported hosting target, but the provided local launcher can run the frontend, Express gateway, Python backend and SQLite database locally. A production deployment would later require independently selected production authentication, database and retention arrangements.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype stores product records, stock movements, physical counts, crate tags, declared quantities, recorded locations, transfers, scan observations, decisions, audit entries and demo-session role/name data in SQLite. It may store user-uploaded JPG, PNG and PDF evidence in private configured storage; attachment metadata and hashes are saved with the related case. Photo-count images are not stored by the photo-count route unless the user separately uploads them as case evidence. When RunPod is configured, the photo, product label and example box are sent to RunPod for inference. The current role selector is a demo mechanism, not real authentication, so the prototype must not be used with sensitive production data. Retention is not yet configurable; local data remains until manually removed.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

Photo counting can miss hidden, stacked, blurred or differently oriented items and may include visually similar objects. OWLv2 depends on a configured RunPod GPU endpoint and may experience cold-start delays, network failures, quota limits or provider errors. The built-in template matcher is less robust when products vary in appearance or scale. The application mitigates these limitations by showing detection boxes, allowing staff to remove false positives or manually correct missed items, requiring confirmation that the photo covers the counting area, and never applying an AI result directly to inventory. A submitted count still undergoes deterministic movement checks, and a manager must approve any stock correction. RFID input is explicitly labelled simulated or entered-ID and is not treated as verified location or quantity evidence.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



KOH HUI YUAN
Team leader - newbies

24 October 2026